

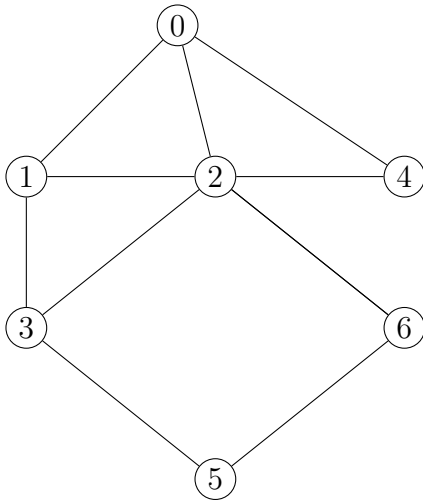
- TD 5 -

Parcours d'un graphe en largeur : BFS (*Breadth First Search*)

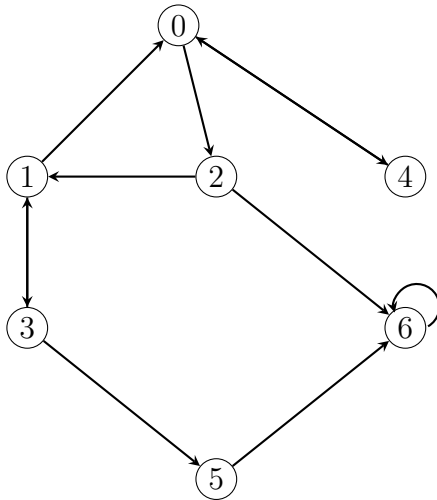
Exercice 1 :

1. On considère les deux graphes ci-dessous :

Graphe G_1 :



Graphe G_2 :



- (a) Déterminer « à la main » la liste des sommets visités à partir du sommet source s , pour $s = 5$, puis $s = 2$, puis pour $s = 1$, à l'aide de l'algorithme de Parcours en largeur.

*A chaque étape de l'algorithme, on donnera la liste des sommets déjà visités depuis s , notée **Access**, et la liste des sommets atteints par l'algorithme mais non encore étudiés, la file **Avisiter**.*

On pourra pour chaque question, remplir un tableau de la forme donnée ci-après.

Dans chaque cas, représenter l'arbre étagé obtenu à l'issu de l'algorithme.

Pour le graphe G_1 et le sommet source $s = 5$:

Étape initiale	Access=[5]	Avisiter=[5]
Étape n° 1 (visite du 1ier sommet)	Access=	Avisiter=
Étape n° 2 (visite du 2ième sommet)	Access=	Avisiter=
Étape n° 3 (visite du 3ième sommet)	Access=	Avisiter=
⋮		

- (b) Définir les deux graphes précédents sous forme d'un dictionnaire.

Vérifier alors vos réponses à l'aide de la commande Python `nx.bfs_tree`.

Vous pourrez vérifier la validité de vos fonctions en utilisant les exemples donnés précédemment (ou tout autre graphe), pour différents sommets sources, et comparer le résultat à celui obtenu par la commande Python `nx.bfs_tree`.

2. (a) Compléter la fonction Python ci-dessous, de paramètres G de type `dict` représentant un graphe (orienté ou non), et s représentant l'un des sommets de G , et renvoyant la liste des sommets accessibles depuis s à l'aide de l'algorithme BFS.

```
def Parcours(G,s):
    Access= ***
    Avisiter= ***
    while *** :
        u=Avisiter.pop()
        L=G[u]
        for k in L:
            if *** :
                ***
                Access.append(k)
    return(Access)
```

- (b) On note K_n le graphe complet d'ordre n , de sommets nommés $0, 1, \dots, n-1$. Représenter graphiquement K_n et faire tourner « à la main » l'algorithme de parcours en largeur de K_n pour un sommet source quelconque. Que constatez-vous ? Pouvez-vous proposer une amélioration à votre fonction `Parcours` ?
- (c) Écrire une fonction en Python nommée `Parcours2`, de paramètres M de type `array` représentant la matrice d'adjacences d'un graphe d'ordre n (orienté ou non) de sommets nommés $0, 1, \dots, n-1$, et s représentant l'un des sommets de G , et renvoyant la liste des sommets accessibles depuis s par un parcours du graphe en largeur.
3. Écrire une fonction réursive, nommée `BFSrec`, de paramètres G , de type `dict` représentant un graphe, `Access` une liste, et `Avisiter` une seconde liste gérée comme une file, qui parcourt le graphe G en largeur, et qui renvoie la liste des sommets accessibles à partir des sommets donnés dans la liste `Avisiter`. Utiliser cette fonction pour retrouver les résultats obtenus dans la question 1.

Exercice 2 : Applications de l'algorithme BFS

Dans cet exercice, on s'appuiera sur l'algorithme écrit dans l'exercice précédent (question 2) : on essaiera de l'adapter afin de répondre aux questions suivantes.

Vous pourrez tester vos fonctions sur les deux exemples de graphes donnés dans l'exercice 1, ou d'autres graphes.

1. Etude de la connexité d'un graphe non orienté

- (a) Soit G un graphe non orienté. Montrer la caractérisation :
 G est connexe si, et seulement si, il existe un sommet s de G tel que tout sommet de G , distinct de s , est accessible à partir s .
- (b) En déduire une fonction Python `Est_connexe`, de paramètre G de type `dict` représentant un graphe non orienté, qui utilise l'algorithme BFS et qui renvoie '`Connexe`' si le graphe G est connexe, et '`Non Connexe`' sinon.

- (c) En vous inspirant de la question précédente, écrire une fonction Python `Nbconnexe`, de paramètre `G` de type `dict` représentant un graphe non orienté, qui utilise l'algorithme BFS et qui renvoie le nombre de composantes connexes du graphe `G`.

2. Recherche des circuits d'un graphe orienté

- (a) Écrire une fonction Python `circ_source`, de paramètres `M` de type `array`, représentant la matrice d'adjacences d'un graphe orienté G dont les sommets sont nommés $0, 1, \dots, n-1$, et `s` représentant l'un des sommets de G , qui utilise l'algorithme BFS et qui renvoie `True` si le graphe G contient un circuit d'origine `s`, et `False` sinon.
- (b) Écrire une fonction Python `circuit`, de paramètre `M` de type `array`, représentant la matrice d'adjacences d'un graphe orienté dont les sommets sont nommés $0, 1, \dots, n-1$, qui utilise la fonction `circ_source` et qui renvoie `True` si le graphe G contient un circuit, et `False` sinon.

3. Recherche des triangles d'un graphe non orienté

Lorsque G est un graphe non orienté, on appelle triangle dans G un cycle élémentaire de longueur 3.

- (a) Ecrire une fonction Python `triangle_source`, de paramètres `G` de type `dict`, représentant un graphe non orienté, et `s` représentant l'un des sommets de G , qui renvoie le nombre de triangles de G dont `s` est l'un des sommets.
- (b) Ecrire une fonction Python `Triangle`, de paramètre `G` de type `dict`, représentant un graphe non orienté, utilisant la fonction `triangle_source` et qui renvoie le nombre de triangles contenus dans G .

4. Étude d'une fonction en Python

On considère la fonction en Python ci-dessous, de paramètre `M` de type `np.array` qui représente la matrice d'adjacences d'un graphe non orienté dont les sommets sont nommés $0, 1, \dots, n-1$.

```

def mystere(M):
    n=len(M)
    Sommet=list(range(n))
    B=True
    Etage=np.zeros(n)
    Avisiter=[]
    while (Sommet!=[] or Avisiter!=[]) and B:
        if Avisiter==[]:
            s=Sommet.pop()
            Avisiter.insert(0,s)
            Etage[s]=1
        s=Avisiter.pop()
        L=M[s,:]
        for u in range(n):
            if L[u]==1:
                if Etage[u]==Etage[s]:
                    B=False
                elif u in Sommet:
                    Sommet.remove(u)
                    Etage[u]=(-1)*Etage[s]
                    Avisiter.insert(0,u)
    if not(B):
        return('False')
    else:
        S1=[]
        S2=[]
        for i in range(n):
            if Etage[i]==1:
                S1.append(i)
            else:
                S2.append(i)
        return(S1,S2)

```

- (a) Définir les matrices ci-dessous en Python, puis exécuter la fonction **mystere** pour ces trois matrices.

$$M_1 = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}, \quad M_2 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad M_3 = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 0 \end{pmatrix}$$

Que renvoie la fonction **mystere** dans chacun des trois cas ?

Représenter les graphes G_1, G_2 et G_3 de matrices d'adjacences respectives M_1, M_2 et M_3 .

- (b) Écrire une fonction en Python **Mat**, de paramètre **n** entier strictement positif, afin qu'elle renvoie la matrice d'adjacences du graphe non orienté de sommets $0, 1, \dots, n-1$ et dont la représentation est le cycle $(0, 1, 2, \dots, n-1, 0)$.

Exécuter la fonction Python `mystere` pour la matrice `Mat(n)`, pour différentes valeurs de l'entier n .

(c) Expliquer le fonctionnement de la fonction `mystere` et ce qu'elle renvoie.

5. Algorithme du plus court chemin

Rappelons que l'instruction `inf=float('inf')` affecte à la variable Python `inf` le symbole mathématique $+\infty$.

(a) Reprendre l'algorithme écrit à la question 2a de l'exercice 1, et le modifier de sorte que la fonction `Parcours`, de paramètres `G` de type `dict` représentant un graphe d'ordre n orienté ou non orienté, de sommets nommés $0, 1, \dots, n-1$, et `s` représentant l'un des sommets de `G`, renvoie deux listes d'ordre n :

- la première, nommée `parents`, donnant en $i^{\text{ième}}$ composante : `inf`, si le sommet i de `G` n'est pas accessible depuis le sommet `s`, et sinon, le numéro du sommet adjacent à (ou du prédécesseur de) i et qui a permis de découvrir le sommet i lors du parcours en largeur du graphe,
- la seconde, nommée `dist`, donnant en $i^{\text{ième}}$ composante : `inf` (représentant $+\infty$), si le sommet i de `G` n'est pas accessible depuis le sommet `s`, et donne sinon le nombre d'arêtes nécessaires en partant du sommet `s` à la découverte du sommet i lors du parcours en largeur du graphe.

(b) Utiliser la fonction précédente, pour écrire une nouvelle fonction Python de paramètres `G` de type `dict` représentant un graphe d'ordre n , de sommets nommés $0, 1, \dots, n-1$, et `s` et `t` représentant deux sommets de `G`, qui renvoie le plus court chemin (ou la plus courte chaîne) d'origine `s` et d'extrémité `t`, sous la forme de la liste des sommets le composant, si un tel chemin existe, et qui renvoie `inf` sinon.
